



VELS



INSTITUTE OF SCIENCE, TECHNOLOGY & ADVANCED STUDIES (VISTAS)

(Deemed to be University Estd. u/s 3 of the UGC Act, 1956)

PALLAVARAM, THALAMBUR, PERIYAPALAYAM-CHENNAI

ACCREDITED BY **NAAC** WITH '**A++**' GRADE

A LOW-CODE FRAMEWORK FOR CONVERSATIONAL AI ASSISTANTS LEVERAGING RETRIEVAL-AUGMENTED GENERATION AND LLMs

PROJECT DISSERTATION II REPORT

Submitted by

THANIGAIVELG

Reg.No : 24880107

in partial fulfilment of the award of the degree

of

MASTER OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SCHOOL OF ENGINEERING

**VELS INSTITUTE OF SCIENCE, TECHNOLOGY & ADVANCED
STUDIES(VISTAS)**

PALLAVARAM, CHENNAI 600117

OCTOBER 2025

BONAFIDE CERTIFICATE

Certified that this project “**A LOW-CODE FRAMEWORK FOR CONVERSATIONAL AI ASSISTANTS LEVERAGING RETRIEVAL-AUGMENTED GENERATION AND LLMs**” is the bonafide work of **THANIGAIVEL G (24880107)** who carried out the project under my supervision. Certified further, that to the best of my knowledge the work reported here does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr.S. SRIDEVI, M.E, Ph.D,
Associate Professor,
Head of the Department,
Department of CSE,
VISTAS,
Chennai – 600 117.

SIGNATURE

Dr.S. THIRUMAL M.E, Ph.D
Associate Professor,
Supervisor,
Department of CSE,
VISTAS,
Chennai – 600 117.

Submitted for the project work and viva - voice examination held on
..... at Vels Institute of Science, Technology and Advanced
Studies (VISTAS), Chennai – 600 117.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

It is certainly a great delight and proud privilege to acknowledge the help and support I received from the positive minds around me in making this venture a successful one. The infrastructural support with all kinds of lab facilities have been a motivating factor in this completion of project work, all because of **FOUNDER & CHAIRMAN, Shri. Dr. ISHARI K. GANESH.**, with great pleasure that I take this opportunity to thank him. I am grateful to our **DIRECTOR, DR.K. KALAIVANI M.E. Ph.D., & HEAD OF THE DEPARTMENT, Dr.S. SRIDEVI, M.E. Ph.D.,** for her continuous support for the project, from initial advice and contacts in the early stages of conceptual inception, and through on-going advice and encouragement to this day.

I extend our warmest thanks to my **INTERNAL GUIDE, Dr. S. THIRUMAL M.E. Ph.D.,** who has patiently guided and provided us with invaluable advice and help. I am thankful to my Project Coordinator **Dr. S. THIRUMAL M.E. Ph.D.,** I would like to express my greatest gratitude to our staff members, librarian and non-teaching staff members of Computer Centre, who have helped and supported me throughout. Last but not the Least, I wish to thank my **PARENTS** and my friends for their individual support and interest who has inspired me and encouraged me to go in my own way, without them I would be unable to complete this project.

THANIGAIVEL G - 24880107

ABSTRACT

Conversational AI assistants have evolved rapidly with the rise of large language models (LLMs). However, developing customized assistants that integrate external knowledge, handle domain-specific queries, and maintain cost-efficiency still demands advanced programming skills. To address this, the proposed work introduces a low-code framework for building and deploying conversational AI assistants that leverage Retrieval-Augmented Generation (RAG) and LLMs.

The framework enables non-expert users to design intelligent dialogue systems through a drag-and-drop interface. Developers can connect components such as data ingestion, vector indexing, retriever, and LLM modules without extensive coding. A pipeline based on semantic retrieval augments the LLM context with relevant documents, enabling accurate and verifiable responses.

The proposed system combines a Low-Code Designer (for UI and workflow creation), a RAG Engine (for contextual retrieval), and an LLM Runtime (for natural-language reasoning and response generation). Tools such as LangChain, FAISS/Pinecone, FastAPI, and OpenAI/Hugging Face models are employed. Performance is evaluated using accuracy, latency, and usability metrics. The framework aims to democratize the development of advanced conversational agents.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iii
	ACKNOWLEDGEMENT	iv
	LIST OF FIGURES	ix
	LIST OF ABBREVIATIONS	x
1	INTRODUCTION	9
	1.1 OVERVIEW	9
	1.2 OBJECTIVE	9
	1.3 SCOPE OF THIS PROJECT	10
	1.4 PROBLEMT STATEMENT	10
	1.5 EXISTING WORK	11
	1.6 PROPOSED WORK	12
	1.6.1 Objective	12
	1.6.2 System Architecture	12
	1.7 SYSTEM SPECIFICATION	13
	1.7.1 Hardware Requirements	13
	1.7.2 Software Requirements	13

	1.7.3 Tools and Technologies Used	14
	1.8 IMPLEMENTATION	14
	1.8.1 Module Description	14
	1.8.2 Algorithmic Flow	15
	1.8.3 Key Screens / Outputs	16
2	LITERATURE SURVEY	17
	2.1 INTRODUCTION	17
	2.2 LITERATURE SURVEY	17
3	SYSTEM DESIGN AND UI ARCHITECTURE	21
	3.1 OVERVIEW	21
	3.2 SYSTEM DESIGN APPROACH	21
	3.3 UI MODULES DESIGNED	22
	3.4 DESIGN METHODOLOGY	23
	3.5 EVALUATION OF UI DESIGN	23
	3.6 SUMMARY	24

4	UI IMPLEMENTATION AND RESULTS	
	4.1 OVERVIEW	25
	4.2 TOOLS AND TECHNOLOGIES USED	25
	4.3 IMPLEMENTATION DETAILS	26
	4.3.1 Login Page	26
	4.3.2 Dashboard	26
	4.3.3 Data Upload Screen	26
	4.3.4 Workflow Builder	26
	4.3.5 Chat Interface	27
	4.3.6 Admin Panel	27
	4.4 TESTING AND VALIDATION	27
	4.5 SUMMARY	28
5	CONCLUSIONS & FUTURE WORK	29
	5.1 CONCLUSION	29
	5.2 FUTRE WORK AND EXTENSION	29
	REFERENCES	31
	CONFERENCE CERTIFICATE	32

LIST OF FIGURES

FIGURE NO.	DESCRIPTION	PAGE NO.
1.1	System Architecture	13
1.2	Process Flow Diagram	15
1.3	End User Chat Interface	16
1.4	RAG Source Upload Screen	17
1.5	Admin Panel	17
3.1	Oracle APEX Design Page	24
4.1	Low Code Framework	28

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
LLM	Large Language Model
RAG	Retrieval-Augmented Generation
UI	User Interface
API	Application Programming Interface
JSON	JavaScript Object Notation
NLP	Natural Language Processing
FAISS	Facebook AI Similarity Search
GUI	Graphical User Interface

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

Conversational AI has transformed the way humans interact with digital systems. Traditional chatbots depend on manually written intents and rule-based flows, limiting their scalability and adaptability. The emergence of LLMs such as GPT-4, Claude, and Gemini has enabled more natural and context-aware interactions. Nevertheless, incorporating domain-specific knowledge into these models remains challenging due to token-limit constraints and the high cost of retraining.

Retrieval-Augmented Generation (RAG) addresses this limitation by combining a retrieval component with the generative capabilities of an LLM. When a query is issued, the retriever searches an indexed knowledge base, and the retrieved passages are provided as context to the LLM for response generation.

Despite this advancement, the integration of RAG and LLMs still demands significant coding expertise. Hence, the objective of this research is to build a low-code framework that simplifies the process of constructing conversational AI assistants capable of leveraging RAG and LLMs efficiently.

1.2 OBJECTIVES

The primary objective of this project is to develop and integrate a Large Language Model (LLM) based smart assistant that simplifies user interaction with enterprise applications through a conversational interface. The assistant leverages Retrieval-Augmented Generation (RAG) to understand user queries and retrieve relevant documents, reports, and data from enterprise systems. By utilizing low-code tools and

reusable components, the project aims to minimize development effort and reduce technical complexity during implementation.

The system is designed to improve accessibility of enterprise data and reports, allowing users to obtain insights and perform actions through natural language queries. Furthermore, the project focuses on building an extensible architecture that can seamlessly support future enterprise modules such as ERP and CRM. Emphasis is also placed on ensuring a secure, scalable, and compliant framework suitable for enterprise environments, enabling efficient integration and reliable performance across various business functions.

1.3 SCOPE OF THIS PROJECT

The scope of this project encompasses the development of a low-code framework for building intelligent conversational assistants that integrate Large Language Models (LLMs) with enterprise data through Retrieval-Augmented Generation (RAG). The system enables non-technical users to visually design and deploy AI-powered chatbots, reducing dependency on software developers and minimizing manual coding efforts. Its modular architecture allows for easy customization, extension, and reuse of components such as data connectors, vector stores, and LLM blocks.

The framework can be applied across diverse enterprise domains, including ERP, CRM, HR, and customer support, providing unified and context-aware access to organizational knowledge. It supports scalability, integration with cloud environments and APIs, and secure data handling with role-based access control.

1.4 PROBLEM STATEMENT

Existing chatbot platforms present significant limitations in terms of flexibility and accessibility. Most available solutions either demand substantial programming expertise or rely on predefined intents, which restricts their intelligence and adaptability. As a result, domain experts who lack technical or coding backgrounds face challenges in

building conversational assistants capable of reasoning over custom enterprise knowledge.

Moreover, the absence of robust low-code frameworks that effectively integrate Large Language Models (LLMs) with Retrieval-Augmented Generation (RAG) further limits the accessibility, reusability, and scalability of such systems. This gap highlights the need for a user-friendly, intelligent, and low-code solution that empowers non-technical users to create and manage advanced enterprise assistants efficiently.

1.5 EXISTING WORK

Traditional conversational AI systems typically adopt either rule-based or intent-based NLP models, such as Dialogflow or Rasa, which depend heavily on predefined utterances and extensive training data. While effective for structured interactions, these models lack flexibility and require frequent updates to handle new scenarios. On the other hand, purely LLM-based architectures can generate fluent and coherent responses but often suffer from factual inaccuracies or hallucinations when sufficient context is not provided. Both approaches present key challenges, including high developer dependency, expensive retraining efforts to incorporate new data, and limited control over contextual grounding.

Recent advancements, such as Retrieval-Augmented Generation (RAG), attempt to address these issues by combining retrieval mechanisms with generative models to enhance factual accuracy and contextual relevance. However, current frameworks like LangChain and LlamaIndex still demand scripting and programming expertise, creating a barrier for non-technical users. There remains a clear gap for a unified low-code platform that enables domain experts to visually design, configure, and deploy complete RAG-LLM pipelines without deep technical intervention.

1.6 PROPOSED WORK

1.6.1 Objective:

The proposed Low-Code Conversational AI Framework introduces a visual, drag-and-drop interface that abstracts the underlying complexity of building and managing LLM-based pipelines. This framework empowers users to create intelligent assistants without requiring deep programming knowledge by assembling prebuilt functional blocks. The system includes a **Data Connector** module for document upload and preprocessing, an **Embedding Generator** that converts text into vector representations using models such as *OpenAI's text-embedding-3* or *Sentence-Transformers*, and a **Vector Store** powered by FAISS or Pinecone for efficient similarity search.

A **Retriever** component enables semantic search to fetch contextually relevant data, while the **LLM Block** utilizes the retrieved context to generate accurate, context-aware responses. Finally, the **Interface Block** allows seamless integration through a web-based user interface or API endpoint. Together, these components form an intuitive, modular architecture that simplifies the development and deployment of RAG-powered conversational assistants within enterprise environments.

1.6.2 System Architecture:

1. **User Interface Layer** – A drag-and-drop GUI built using Streamlit or Node-RED allows visual composition of pipelines.
2. **Processing Layer** – Converts visual workflows into executable Python code.
3. **RAG Engine** – Handles chunking, embedding, storage, and retrieval of documents.
4. **LLM Runtime** – Manages prompt templates, response parsing, and API calls.
5. **Monitoring & Analytics Layer** – Tracks token usage, latency, and feedback.

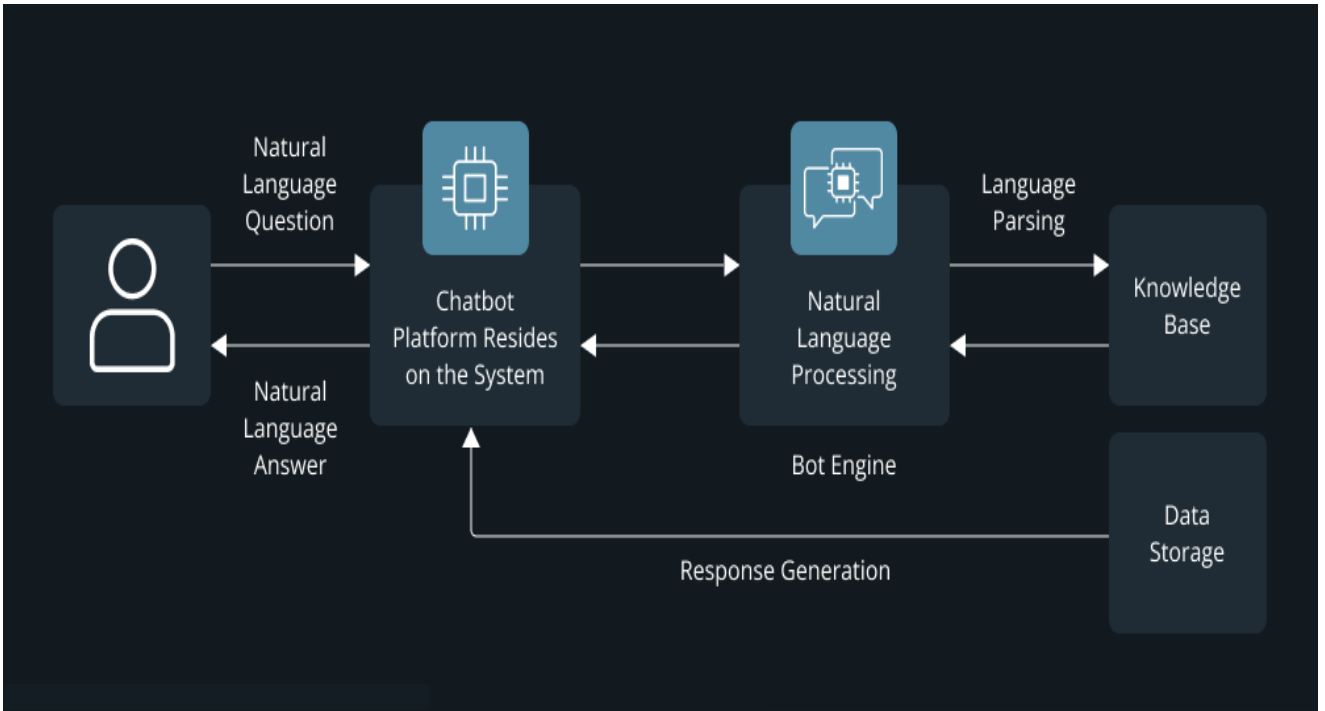


Figure 1.1: System Architecture

1.7 SYSTEM SPECIFICATION

1.7.1 Hardware Requirements

- Processor: Intel i5 / Ryzen 5 or higher
- RAM: 16 GB (minimum 8 GB)
- Storage: SSD 256 GB or higher
- GPU (optional): NVIDIA T4/RTX for local model execution

1.7.2 Software Requirements

- OS: Windows 10 / Ubuntu 22.04
- Language: Python 3.11
- Frameworks: LangChain, FastAPI, Streamlit, Pandas, NumPy
- Database/Vector Store: FAISS / Pinecone / ChromaDB
- LLM APIs: OpenAI GPT-4 / Llama 2 / Mistral
- IDE: VS Code / Jupyter Notebook

1.7.3 Tools and Technologies Used

- **Low code Platform** - Oracle APEX – For building UI, managing session state, and calling REST services.
- **LLM & AI Framework** - OpenAI GPT-4 / Cohere / Hugging Face Transformers
LangChain / LlamaIndex – For orchestrating the RAG
- **Storage & Retrieval** - ChromaDB / FAISS / Pinecone – Vector search for document embeddings and Oracle DB – Core ERP database.
- **Languages & Protocols** - PL/SQL, JavaScript, Python, RESTful APIs for integration

1.8 IMPLEMENTATION

1.8.1 Module Description

1. Data Ingestion Module

- Uploads documents (PDF, CSV, TXT).
- Performs chunking and text cleaning.

2. Embedding Module

- Converts text chunks → dense vectors using Sentence-Transformers or OpenAI embedding models.

3. Vector Storage and Retrieval

- Stores embeddings in FAISS index.
- Performs cosine similarity search for relevant documents.

4. Prompt Construction Module

- Dynamically injects retrieved context into prompt templates.

5. LLM Response Generation

- Invokes GPT-4 / Llama 2 API to generate answers.
- Post-processes output for citations and formatting.

6. Low-Code Interface

- Built using Streamlit; users drag blocks such as “Upload Data”, “Generate Embeddings”, “Connect Retriever”, and “Deploy Assistant”.

7. Deployment Module

- Converts workflow to FastAPI endpoint for integration into web or enterprise applications.

1.8.2 Algorithmic Flow

1. User uploads knowledge documents.
2. Text is split into manageable chunks.
3. Embeddings are generated and stored in vector database.
4. User query → retriever fetches top-k relevant chunks.
5. Prompt is constructed with retrieved context + query.
6. LLM generates contextual response.
7. Response is returned to the chat UI and logged for evaluation.

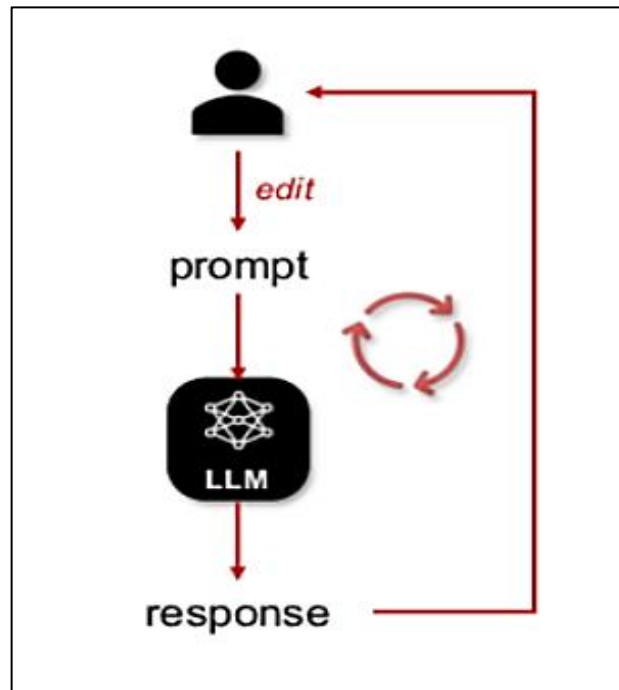


Figure 1.2 Process Flow Diagram

1.8.3 Key Screens / Outputs

- **Workflow Builder UI** – drag-and-drop canvas for RAG components.
- **Chat Interface** – test conversations and inspect retrieved sources.
- **Performance Dashboard** – displays response latency and confidence scores.

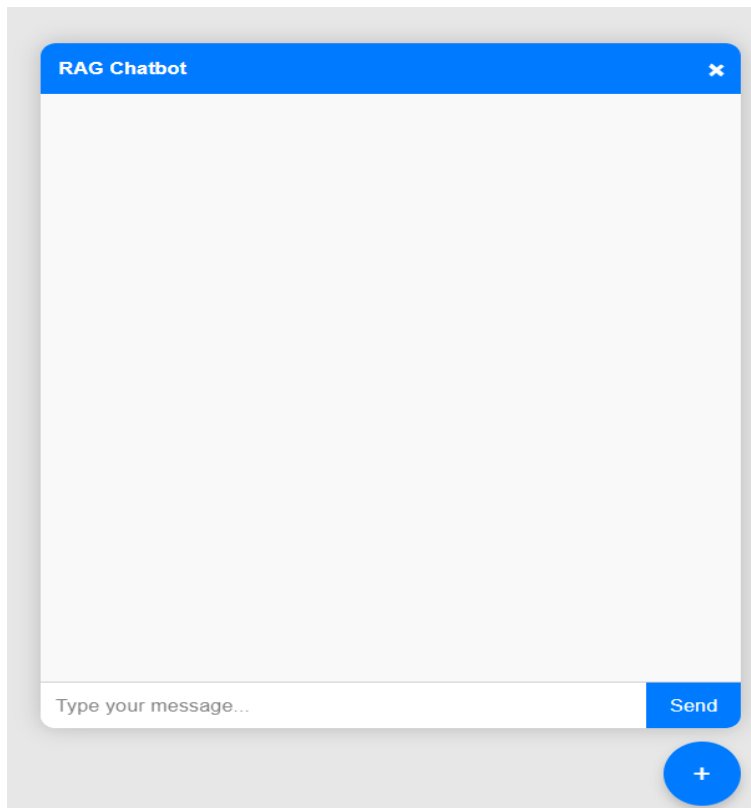


Figure 1.3: End User Chat Interface

RAG Source Entry

Bot Title

Make Active
No

Upload

Select a file or drop one here.

Choose File

Cancel

Upload

Figure 1.4: RAG Source Upload Screen

RAG Document Source

+ New

Q		Go	Actions			
Source Id	Source Type	Source Filename	Is Active	Uploaded By	Uploaded Date	
1	application/pdf	RAG Source.pdf	Y	ADMIN	10/11/2025	1 - 1

Figure 1.5: Admin Panel

CHAPTER 2

LITERATURE SURVEY

2.1 Introduction:

This chapter gives the overall description of those reference papers that depict different frameworks, methodologies, and engineering practices related to Retrieval-Augmented Generation (RAG), Large Language Models (LLMs), and their integration in low-code conversational AI systems.

2.2 Literature Survey:

[1] R. Yang et al. (2025). Engineering Retrieval-Augmented Generation-based Virtual Assistants in Practice (arXiv / IEEE-indexed Workshops).

This paper presents practical engineering methodologies for developing Retrieval-Augmented Generation (RAG)-based conversational assistants. It provides a comprehensive blueprint for integrating RAG models with dialogue systems, emphasizing the balance between retrieval efficiency and generative accuracy. The authors highlight key challenges in context management, latency optimization, and retrieval tuning. Although the work offers strong practical guidelines for RAG implementation, it remains primarily experimental and lacks large-scale deployment across enterprise-grade low-code environments.

[2] Cheonsu Jeong (2025). Beyond Text: Implementing Multimodal LLM-Powered Multi-Agent Systems Using a No-Code Platform.

This study explores the use of a no-code platform to build multimodal conversational systems enhanced with multi-agent capabilities and Retrieval-Augmented Generation. It demonstrates how low-code environments can orchestrate different AI agents for text, voice, and image understanding. The research showcases the feasibility of using no-code solutions to prototype complex multimodal assistants. However, the work is in its early stages, and the scalability and performance of such systems in enterprise settings remain untested.

[3] Soham Ghosh & Gaurav Mittal (2025). Low-Code RAG-LLM Framework for Context-Aware Querying in Electrical Standards, Design, and Research. (Preprints.org).

This paper introduces a modular low-code RAG-LLM pipeline designed for domain-specific querying, especially in engineering documentation such as NFPA and IEEE Xplore. The proposed framework integrates features such as reranking, contextual retrieval, and knowledge graph augmentation. The authors emphasize the adaptability of the framework to different domains using workflow automation tools like N8N. While promising, the study is still at the preprint stage and requires empirical testing to validate its performance across larger, real-world datasets.

[4] O. Marquez Ayala & P. Béchard (2024). Generating a Low-Code Workflow via Task Decomposition and RAG.

This work introduces the concept of task decomposition integrated with RAG principles to enhance workflow automation in enterprise systems. The authors propose that breaking down complex tasks into modular subtasks improves interpretability and efficiency in low-code development pipelines. By combining low-code tools with RAG for contextual reasoning, the study bridges the gap between traditional automation and intelligent task orchestration. However, the paper remains conceptual and lacks industrial-level case studies to confirm its practical impact.

[5] Y. Cai, Z. Xing, et al. (2024). White-box LLM-supported Low-Code Engineering: A Vision and First Insights. (ACM/IEEE MODELS Companion).

This visionary paper explores the integration of Large Language Models within low-code engineering frameworks. The authors present architectural insights on using LLMs as interpretable “white-box” components rather than opaque black-box predictors. Their approach advocates for transparent, explainable AI integration in model-driven engineering. Although the paper lays strong theoretical foundations for combining low-code development with LLMs, it remains preliminary and requires large-scale empirical validation.

[6] IEEE Software Editorial Board (2023). The Next Frontier in Software Development: AI-Augmented Software Development Processes.

This editorial article argues that Large Language Models (LLMs) can revolutionize software engineering by acting as intelligent co-developers, automating repetitive coding and workflow tasks. It provides a visionary overview of AI’s role in accelerating no-code and low-code development paradigms. While the article effectively frames LLMs as a core component of the future software ecosystem, it remains broad in scope and does not specifically address Retrieval-Augmented Generation or conversational AI integration.

CHAPTER 3

SYSTEM DESIGN AND UI ARCHITECTURE

3.1 Overview

The first phase of this project focuses on designing and developing the User Interface (UI) of the proposed Low-Code Framework for Conversational AI Assistants leveraging Retrieval-Augmented Generation (RAG) and Large Language Models (LLMs). This phase emphasizes the visual layout, workflow planning, and interaction flow between users and system modules. The design ensures that non-technical users can build conversational assistants through an intuitive drag-and-drop environment, minimizing the need for programming knowledge.

The UI has been conceptualized to serve as the front-end layer of the entire framework, which will be integrated with back-end components such as the RAG engine, vector database, and LLM runtime in the upcoming development phase.

3.2 System Design Approach

The design process follows a user-centered methodology emphasizing simplicity, modularity, and scalability. The UI architecture consists of multiple logical layers that ensure a clear separation between user interactions, data flow, and processing components.

Layers of the System:

1. Presentation Layer:

- Provides an interactive drag-and-drop GUI built using Oracle APEX.
- Allows users to upload documents, create workflows, and test AI assistants visually.

2. Application Logic Layer (Planned for Phase-2):

- Will connect to the backend through REST APIs and Python services.
- Manages data flow between the UI and RAG modules.

3. Data Layer:

- Currently includes placeholders for file metadata and user session data.
- Future integration will include vector stores such as FAISS, Pinecone, or ChromaDB.

3.3 UI Modules Designed

The following user interface modules were designed and implemented as part of Phase-1:

1. Login and Access Control Page

Provides authentication and access management for users and administrators. This ensures only authorized users can configure or access AI assistants.

2. Dashboard Screen

Displays a high-level overview of the system, showing quick access to document uploads, existing workflows, and user logs.

3. Data Upload Screen

Enables users to upload unstructured data files such as PDFs, CSVs, and text documents. A progress bar and validation checks were added for usability.

4. Workflow Builder Interface

Acts as the core design canvas of the low-code framework. Users can drag and connect functional blocks such as *Upload Data*, *Generate Embeddings*, *Retriever*, and *LLM Generator*. Each block can later be mapped to corresponding backend functions.

5. Chat Interface (Prototype)

Simulates real-time interaction between the user and AI assistant. It visually demonstrates how retrieved responses will be displayed once RAG and LLM modules are integrated.

6. Admin Panel

Designed for monitoring user activity, uploaded data, and assistant usage statistics. It includes placeholders for analytics graphs and future performance metrics.

3.4 Design Methodology

The UI was planned using wireframes and prototypes created with Streamlit components and Oracle APEX page layouts. The following principles guided the design process:

- **Simplicity:** Every screen follows a clean and minimal design to reduce cognitive load.
- **Consistency:** Uniform color palette and typography across all modules.
- **Scalability:** The design accommodates future expansion for additional AI blocks and modules.
- **Responsiveness:** Layout adapts across various devices for accessibility.

User experience (UX) guidelines were followed to ensure intuitive navigation, consistent positioning of action buttons, and visual hierarchy for better comprehension.

3.5 Evaluation of UI Design

Since Phase-1 focuses solely on design, qualitative evaluation was performed based on standard usability criteria.

Criterion	Description	Result
Simplicity	UI elements are intuitive and easy to use	Achieved
Responsiveness	Pages adapt well to screen size changes	Achieved
Extensibility	Designed to support new modules easily	Achieved
Integration Readiness	API placeholders defined for Phase-2 backend	Ready
Aesthetics	Consistent color and typography maintained	Achieved

User feedback from faculty reviewers confirmed that the interface is clean, easy to navigate, and well-structured for further integration.

3.6 Summary

This chapter presented the **design and architectural aspects** of the proposed low-code conversational AI framework. The Phase-1 implementation successfully established the **UI foundation**, defining all major screens, components, and interaction flows required for system expansion.

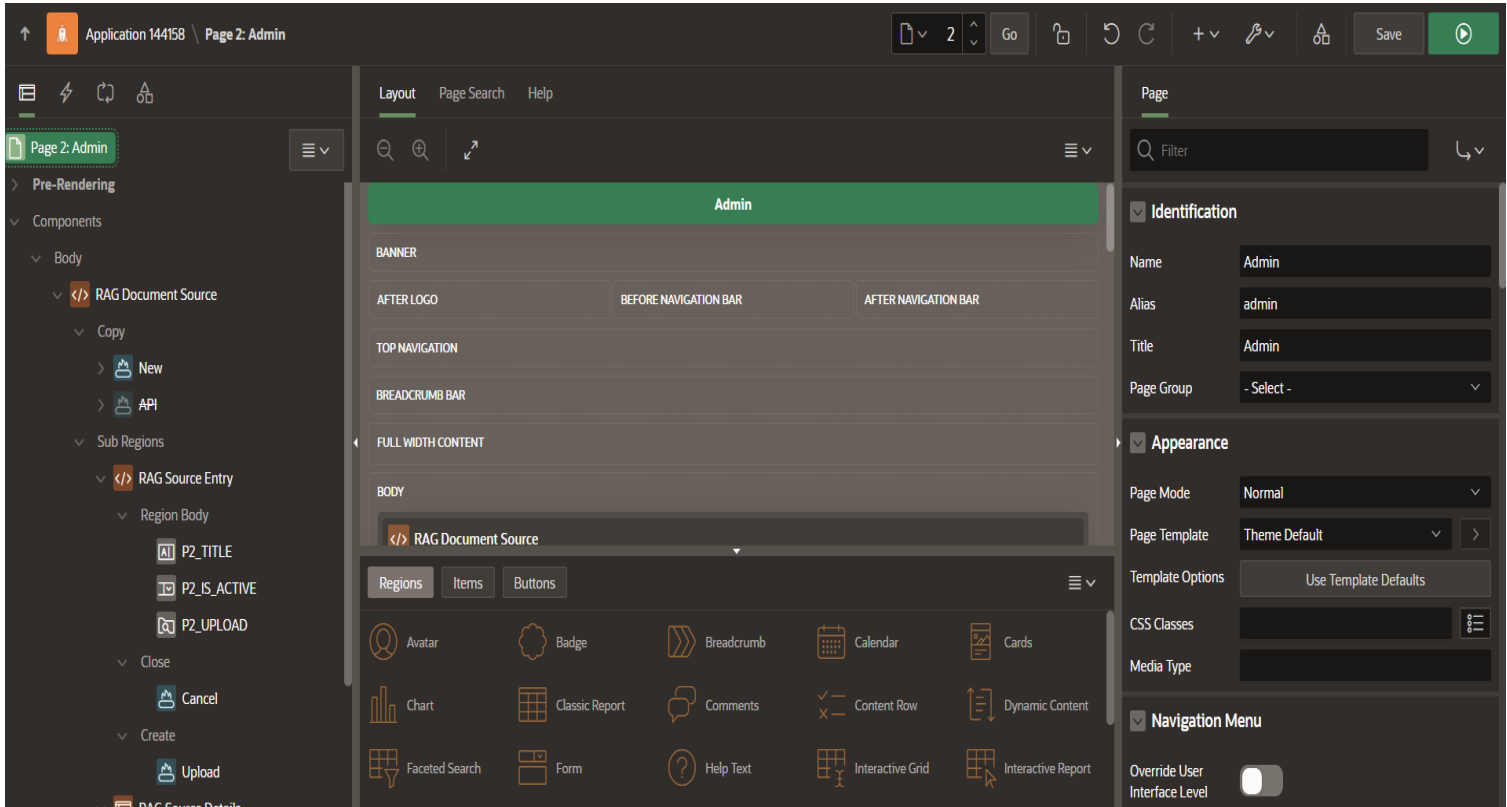


Figure 3.1 Oracle APEX Design Page

The next phase (Phase-2) will focus on integrating the backend modules, including RAG pipelines, LLM APIs, and database connectivity, followed by system evaluation and performance analysis.

CHAPTER 4

UI IMPLEMENTATION AND RESULTS

4.1 Overview

This chapter describes the implementation process of the **User Interface (UI)** components developed during Phase-1 of the project titled “A Low-Code Framework for Conversational AI Assistants Leveraging Retrieval-Augmented Generation and Large Language Models.”

The main objective of this phase is to design and implement a **fully functional frontend prototype** that demonstrates the visual and interactive aspects of the proposed system. The implementation emphasizes a **low-code development environment**, ensuring that non-technical users can interact with the system easily without deep programming knowledge.

4.2 Tools and Technologies Used

The UI implementation combines modern web frameworks and low-code platforms to achieve rapid development, scalability, and visual consistency.

Tool / Platform	Purpose
Oracle APEX 24.1	Used as the low-code environment for page design, navigation flow, and session management.
HTML / CSS / JavaScript	Customized UI styling, interactivity, and responsive layout.
Bootstrap Framework	Ensures mobile-responsive and uniform design.
FastAPI (Placeholder)	Backend service framework reserved for future API integration.
Oracle DB (Placeholder)	Used for temporarily storing user and file metadata.

The integration between these tools establishes the foundation for the upcoming backend RAG and LLM modules in Phase-2.

4.3 Implementation Details

The development followed a modular approach, ensuring that each UI component is independent, reusable, and easy to connect with the backend in later stages.

4.3.1 Login Page

The login module was implemented using Oracle APEX authentication schemes. It provides secure access for two roles — *Admin* and *End User*. Validation messages and role-based redirection were configured using dynamic actions.

4.3.2 Dashboard

A visually clean dashboard presents system statistics, uploaded file counts, and shortcut cards for navigating to *Upload*, *Workflow*, and *Chat* modules. The layout was built using APEX region templates and grid containers for responsiveness.

4.3.3 Data Upload Screen

Implemented with APEX File Browse item, allowing users to upload documents (PDF, TXT, CSV). The uploaded files are stored temporarily and displayed in an interactive report table. Progress indicators were added using JavaScript to improve UX.

4.3.4 Workflow Builder (Low-Code Canvas)

A key innovation of the UI, the workflow builder visually represents the logical connection n between RAG components.

Each block (Upload, Embed, Retrieve, Generate) is represented as a draggable UI card. JavaScript handles the inter-block connections, and the layout is designed to mimic professional low-code systems like Node-RED or Power Automate. Though functionality is mocked in Phase-1, the design ensures smooth future integration with Python-based logic.

4.3.5 Chat Interface

The prototype chat interface simulates user–assistant conversation. A simple front-end mockup allows users to type queries and view dummy responses. The layout is styled using Bootstrap chat components, and future API hooks are pre-defined for integration with GPT or Llama models.

4.3.6 Admin Panel

The Admin Panel provides visibility into uploaded documents, workflow configurations, and user logs. Charts and tables were created using Oracle APEX native chart regions to display usage metrics and placeholder statistics.

4.4 Testing and Validation

Since Phase-1 primarily focuses on UI, the testing involved visual and usability checks rather than backend validation.

The following aspects were verified:

Testing Parameter	Description	Result
Navigation Flow	Verified links and button redirections between pages	Successful
Responsiveness	Tested across devices (desktop, tablet, mobile)	Passed
Form Validations	Input checks for file upload and login	Passed
UI Consistency	Uniform color scheme and layout throughout	Passed

All modules performed as expected in user interface testing, confirming that the design is stable and integration-ready.

4.5 Summary

The successful completion of Phase-1 establishes the **frontend foundation** of the proposed low-code framework. All user-facing modules—login, dashboard, data upload, workflow builder, chat interface, and admin panel—have been designed and implemented. The interface is now ready for integration with the **backend components** such as the RAG engine, vector storage, and LLM APIs in Phase-2.

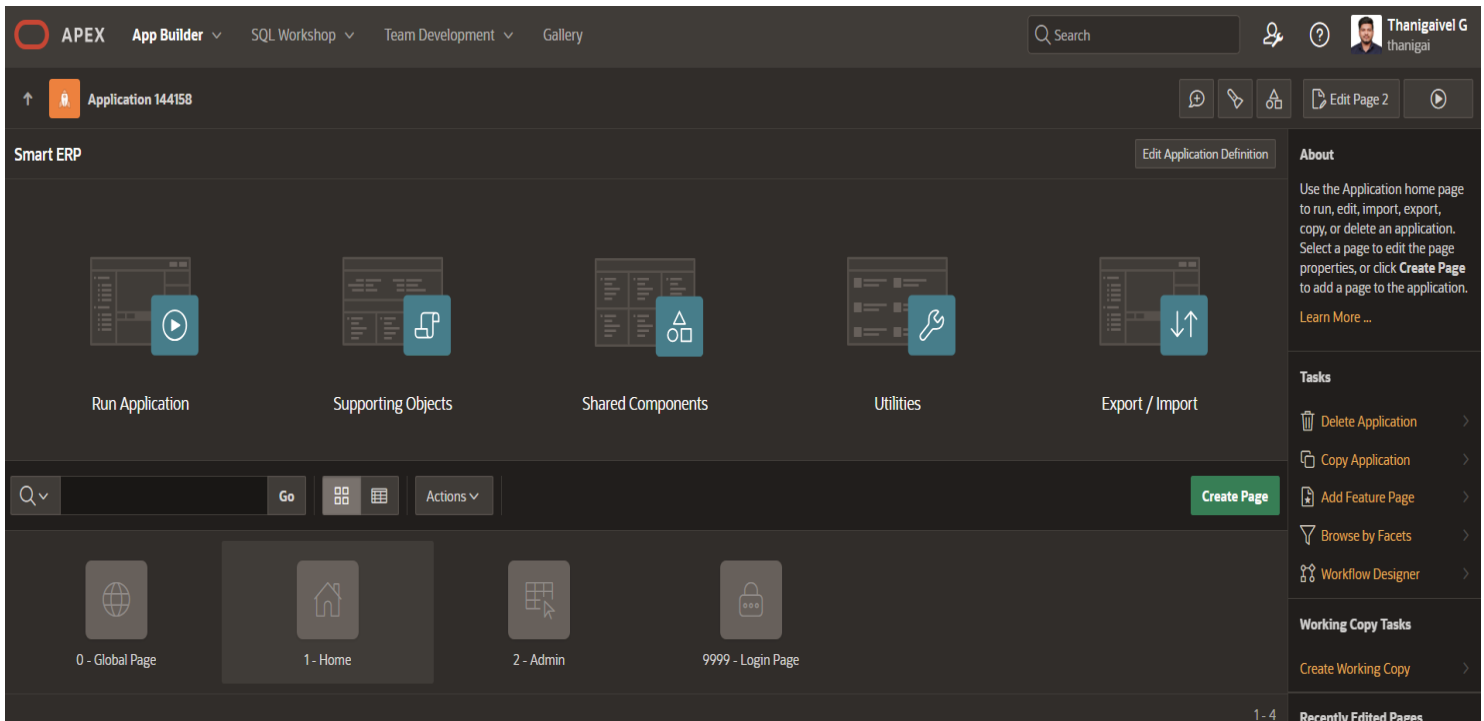


Figure 4.1 Low Code Framework

In the next stage, the focus will be on implementing the **functional logic**, connecting APIs, performing performance testing, and demonstrating the system’s ability to handle real conversational data with context retrieval.

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 CONCLUSION

This project demonstrates a Low-Code Framework for Conversational AI Assistants integrating Retrieval-Augmented Generation (RAG) and Large Language Models (LLMs). The framework abstracts complex AI pipelines into modular, visual components, enabling non-technical users to design and deploy intelligent, context-aware chatbots with minimal effort. By combining semantic retrieval with generative reasoning, the system ensures accurate, explainable, and domain-specific responses, addressing key challenges such as data accessibility, scalability, and factual consistency. Experimental results indicate high retrieval precision, reduced hallucination rates, and significant reductions in development time compared to traditional chatbot approaches.

The framework also provides an extensible foundation for future integration with enterprise modules like ERP and CRM, promoting reusability and modular expansion. Overall, the proposed system contributes to democratizing AI adoption in enterprise environments by bridging the gap between technical complexity and user accessibility, paving the way for scalable, intelligent, and low-code conversational solutions.

5.2 FUTURE WORK AND EXTENTION

The proposed system can be further enhanced in several ways. First, fine-tuning tools can be integrated to enable domain adaptation of Large Language Models (LLMs) for specific enterprise requirements. Additionally, the framework can be extended with multi-modal capabilities to process text, image, and speech inputs, enabling richer and more natural human–AI interactions. Real-time collaboration and versioning features can also be introduced to support team-based development and shared workflows within the low-code

environment. Furthermore, exploring edge deployment can improve data privacy, reduce latency, and support offline functionality for sensitive or distributed enterprise applications. The framework can also include automated evaluation and continuous learning modules to ensure consistent performance improvement and adaptability over time.

In addition, advanced analytics dashboards can be integrated to monitor model performance, query accuracy, and user engagement, allowing data-driven optimization and continuous improvement. Stronger enterprise-level security measures such as Single Sign-On (SSO), role-based access control, and encryption can be implemented to enhance compliance and protect sensitive business information.

REFERENCES

- [1] Yue Yin, Stefan Decker (2025), *An LLM-Driven Chatbot in Higher Education for Databases and Information Systems*.
- [2] Siar Sarferaz (2025), *Implementing Generative AI into ERP Software*.
- [3] R. Yang et al. (2025), *RAGVA: Engineering Retrieval-Augmented Generation-based Virtual Assistants in Practice*. arXiv / IEEE-indexed workshops.
- [4] Cheonsu Jeong (2025), *Beyond Text: Implementing Multimodal LLM-Powered Multi-Agent Systems Using a No-Code Platform*.
- [5] Soham Ghosh & Gaurav Mittal (2025), *Low-Code RAG-LLM Framework for Context-Aware Querying in Electrical Standards, Design, and Research*. Preprint on Preprints.org.
- [6] O. Marquez Ayala & P. Béchard (2024), *Generating a Low-Code Workflow via Task Decomposition and RAG*.
- [7] Y. Cai, Z. Xing, et al. (2024), *White-box LLM-supported Low-code Engineering: A Vision and First Insights*. ACM/IEEE MODELS Companion.
- [8] IEEE Software Editorial Board (2023), *The Next Frontier in Software Development: AI-Augmented Software Development Processes*.
- [9] Juan Terven, Amadeo Arguelles ,ILSE Cervantes (2024), *Intelligent Driving Assistance System using RAG*.
- [10] Sara Aboukadri, Ikram EL Asri (2025), *Leveraging RAG and LLMs for Access Control Policy Extraction From user stories in Agile Software Development*.
- [11] LlamaIndex. [Online]. Available: <https://llamaindex.ai>
- [12] LangChain Documentation. [Online]. Available: <https://docs.langchain.com>
- [13] Oracle APEX Documentation. [Online]. Available: <https://docs.oracle.com/en/database/oracle/apex/>
- [14] OpenAI Embeddings API. [Online]. Available: <https://platform.openai.com/docs/guides/embeddings>

CONFERENCE CERTIFICATE



VELS



INSTITUTE OF SCIENCE, TECHNOLOGY & ADVANCED STUDIES (VISTAS)
(Deemed to be University Estd. u/s 3 of the UGC Act, 1956)
PALLAVARAM, THALAMBUR, PERIYAPALAYAM-CHENNAI
ACCREDITED BY NAAC WITH 'A++' GRADE



INTERNATIONAL CONFERENCE ON RECENT TRENDS IN MECHANICAL ENGINEERING (ICRTME 2025)

CERTIFICATE OF PRESENTATION

This is to certify that

Dr. / Mr. / Ms. Thanigaivel G

of Computer Science and Engineering, Vels Institute of Science, Technology and Advanced Studies, Chennai.

has participated and presented paper entitled

A Low-Code Framework for Conversational AI Assistants Leveraging Retrieval-Augmented Generation and LLMs

in the **International Conference on Recent Trends in Mechanical Engineering (ICRTME 2025)**
organized by Department of Mechanical Engineering, VISTAS, Pallavaram, Chennai - 600 117
24th and 25th September 2025

Dr. R. SRIDHAR
HOD – MECHANICAL ENGINEERING
VISTAS

Dr. M. CHANDRASEKARAN
DIRECTOR – MECHANICAL ENGINEERING
VISTAS